

PPO Portfolio Optimization - Intrinsic Reward Ablation

August 7, 2026

1 PPO Portfolio Optimization: Paper-Faithful, Robust, and Legacy Surprise

This experiment compares alternative Surprise reward definitions using the exact **real-data PPO control** from `FinRL_PortfolioOptimization_PPO_Synthetic_vs_Real`.

All variants use the same real data, per-ticker price scaling, 50-day EIIE observation, `MultiInputPolicy`, PPO hyperparameters, **0.25% transaction cost**, seeds, validation schedule, and external-reward backtest. The reward supplied to PPO during training is the only experimental difference.

1.1 Experimental protocol

The half-open periods and training settings match the Real-vs-Synthetic experiment:

Block	Period	Purpose
Train	2024-01-01 to 2024-10-01	Fit the policy and real-data scaler
Validation	2024-10-01 to 2025-01-01	Select checkpoints using external reward
Test	2025-01-01 to 2026-01-01	Final untouched real-data comparison

Only the original `with_fee=0.0025` condition is run. Formal mode therefore contains **6 reward variants** $\times$ **1 commission setting** $\times$ **3 seeds** = **18 runs**. Validation and every reported financial metric use the original scaled log-return reward only. Intrinsic reward is training-only.

```
[1]: from __future__ import annotations

import glob
import os
import sys
import warnings
from pathlib import Path

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
```

```

import torch
from IPython.display import Image as NotebookImage
from stable_baselines3 import PPO
from tqdm.auto import tqdm

def _bootstrap_project_root(start: Path) -> Path:
    for candidate in (start.resolve(), *start.resolve().parents):
        if (candidate / "finrl").is_dir() and (candidate / "examples").is_dir():
            return candidate
    raise RuntimeError("Could not locate the FinRL project root.")

PROJECT_ROOT = _bootstrap_project_root(Path.cwd())
if str(PROJECT_ROOT) not in sys.path:
    sys.path.insert(0, str(PROJECT_ROOT))

from finrl.experiments.notebook_utils import check_calendar_coverage
from finrl.experiments.notebook_utils import environment_flag
from finrl.experiments.notebook_utils import read_real_ohlcvs_csvs
from finrl.experiments.portfolio_intrinsic import evaluate_portfolio_periods
from finrl.experiments.portfolio_intrinsic import plan_portfolio_training
from finrl.experiments.portfolio_intrinsic import portfolio_environment_kwargs
from finrl.experiments.portfolio_intrinsic import portfolio_ppo_kwargs
from finrl.experiments.portfolio_intrinsic import prepare_real_portfolio_periods
from finrl.experiments.portfolio_intrinsic import resolve_variant_reward_config
from finrl.experiments.portfolio_intrinsic import train_portfolio_variant
from finrl.experiments.portfolio_intrinsic import VARIANT_WEIGHTS
from finrl.experiments.stock_intrinsic import plot_intrinsic_diagnostics
from finrl.experiments.stock_intrinsic import plot_learning_curves_by_variant
from finrl.experiments.stock_intrinsic import plot_period_equity_comparison
from finrl.experiments.stock_intrinsic import plot_sharpe_seed_selection
from finrl.experiments.stock_intrinsic import plot_test_metric_comparison
from finrl.experiments.stock_intrinsic import plot_test_portfolio_weights
from finrl.meta.env_portfolio_optimization.env_portfolio_optimization_gymnasium_
import (
    PortfolioOptimizationGymnasiumEnv,
)

warnings.filterwarnings("once")
DEVICE = os.getenv("FINRL_DEVICE", "cpu")
torch.set_num_threads(int(os.getenv("FINRL_TORCH_NUM_THREADS", "1")))
print("project root:", PROJECT_ROOT)
print("device:", DEVICE)

```

WARNING: CPU random generator seem to be failing, disabling hardware random number generation

WARNING: RDRND generated: 0xffffffff 0xffffffff 0xffffffff 0xffffffff

Gym has been unmaintained since 2022 and does not support NumPy 2.0 amongst other critical functionality.
Please upgrade to Gymnasium, the maintained drop-in replacement of Gym, or contact the authors of your software and request that they upgrade.
Users of this version of Gym should be able to simply replace 'import gym' with 'import gymnasium as gym' in the vast majority of cases.
See the migration guide at https://gymnasium.farama.org/introduction/migration_guide/ for additional information.
/opt/anaconda3/envs/finrl_exp/lib/python3.10/site-packages/tqdm/auto.py:21: TqdmWarning: IProgress not found. Please update jupyter and ipywidgets. See https://ipywidgets.readthedocs.io/en/stable/user_install.html
from .autonotebook import tqdm as notebook_tqdm
project root: /home/cwei1213/FinRL_Exp
device: cpu

1.2 1. Configuration

```
[2]: REAL_DATA_GLOB = os.getenv("FINRL_REAL_DATA_GLOB", "data/real/*.csv")
RESULT_ROOT = Path(
    os.getenv(
        "FINRL_INTRINSIC_RESULT_ROOT",
        str(PROJECT_ROOT / "results" /
↪ "ppo_portfolio_intrinsic_reward_ablation"),
    )
).resolve()

TRAIN_START, TRAIN_END = "2024-01-01", "2024-10-01"
VALIDATION_START, VALIDATION_END = "2024-10-01", "2025-01-01"
TEST_START, TEST_END = "2025-01-01", "2026-01-01"
EXPECTED_TICKERS = ["AAPL", "AMZN", "GOOGL", "MSFT", "NVDA"]
PRICE_COLUMNS = ["close", "high", "low"]
TIME_WINDOW = 50
INITIAL_AMOUNT = 100_000
REWARD_SCALING = 100.0

RUN_MODE = os.getenv("FINRL_INTRINSIC_RUN_MODE", "full").lower()
RUN_SETTINGS = {
    "smoke": {
        "seeds": [0, 1],
        "total_timesteps": 4_096,
        "eval_freq": 2_048,
    },
    "full": {
        "seeds": [0, 1, 2],
        "total_timesteps": 200_000,
        "eval_freq": 20_000,
```

```

    },
}
if RUN_MODE not in RUN_SETTINGS:
    raise ValueError(f"RUN_MODE must be one of {sorted(RUN_SETTINGS)}")

SEEDS = RUN_SETTINGS[RUN_MODE]["seeds"]
TOTAL_TIMESTEPS = RUN_SETTINGS[RUN_MODE]["total_timesteps"]
EVAL_FREQ = RUN_SETTINGS[RUN_MODE]["eval_freq"]
if os.getenv("FINRL_INTRINSIC_SEEDS"):
    SEEDS = [int(value) for value in os.environ["FINRL_INTRINSIC_SEEDS"].
        ↪split(",")]
TOTAL_TIMESTEPS = int(
    os.getenv("FINRL_INTRINSIC_TOTAL_TIMESTEPS", str(TOTAL_TIMESTEPS))
)
EVAL_FREQ = int(os.getenv("FINRL_INTRINSIC_EVAL_FREQ", str(EVAL_FREQ)))
COMMISSION_CONFIGS = {"with_fee": 0.0025}
VARIANTS = list(VARIANT_WEIGHTS)
WARMUP_STEPS = int(os.getenv("FINRL_INTRINSIC_WARMUP_STEPS", "1024"))
REPLAY_CAPACITY = int(os.getenv("FINRL_INTRINSIC_REPLAY_CAPACITY", "100000"))
PPO_KWARGS = portfolio_ppo_kwargs(DEVICE)
for env_name, key in (
    ("FINRL_PPO_N_STEPS", "n_steps"),
    ("FINRL_PPO_BATCH_SIZE", "batch_size"),
    ("FINRL_PPO_N_EPOCHS", "n_epochs"),
):
    if os.getenv(env_name):
        PPO_KWARGS[key] = int(os.environ[env_name])

ARTIFACT_ROOT = RESULT_ROOT / RUN_MODE
TRAIN_CACHE_ROOT = RESULT_ROOT / "train_cache"
USE_TRAIN_CACHE = environment_flag("FINRL_USE_TRAIN_CACHE", True)
FORCE_RETRAIN = environment_flag("FINRL_FORCE_RETRAIN", False)
ARTIFACT_ROOT.mkdir(parents=True, exist_ok=True)
TRAIN_CACHE_ROOT.mkdir(parents=True, exist_ok=True)

print("run mode:", RUN_MODE)
print("variants:", VARIANTS)
print("commission settings:", COMMISSION_CONFIGS)
print("seeds:", SEEDS)
print("timesteps per run:", f"{TOTAL_TIMESTEPS:,}")
print("validation frequency:", f"{EVAL_FREQ:,}")
print("formal run count:", len(VARIANTS) * len(COMMISSION_CONFIGS) * len(SEEDS))
print("training cache:", "enabled" if USE_TRAIN_CACHE else "disabled")
print("force retrain:", FORCE_RETRAIN)
print("output:", ARTIFACT_ROOT)

```

run mode: full

variants: ['baseline', 'paper_faithful', 'robust_surprise', 'surprise',

```

'dejavu', 'combined']
commission settings: {'with_fee': 0.0025}
seeds: [0, 1, 2]
timesteps per run: 200,000
validation frequency: 20,000
formal run count: 18
training cache: enabled
force retrain: False
output:
/home/cwei1213/FinRL_Exp/results/ppo_portfolio_intrinsic_reward_ablation/full

```

The common training reward is

$$r_t = r_t^{ext} + \eta_t \left(\alpha \hat{r}_t^{surprise} + \beta \hat{r}_t^{dejavu} \right).$$

The six variants are:

Variant	Surprise definition	Nominal α	β
baseline	none	0	0
paper_faithful	summed Gaussian NLL; no ReLU/RMS/bonus clipping	0.05	0
robust_surprise	positive running z-score of mean Gaussian NLL	0.05	0
surprise	current/legacy positive-RMS implementation	0.05	0
dejavu	reconstruction MSE only	0	0.05
combined	legacy Surprise + Déjà vu	0.05	0.05

For `paper_faithful`, the effective α is divided by observation dimension. With the current $3 \times 5 \times 50 + 6 = 756$ dimensional input, this gives $0.05 / 756$, preventing the summed NLL from receiving a 756-fold reward-scale advantage. This changes only an unspecified hyperparameter scale; the Surprise score itself remains formula-faithful.

Every episode starts fully in cash. The policy outputs six continuous actions—cash plus five stocks—which the unchanged parent environment scales and softmaxes into portfolio weights.

1.3 2. Load, validate, split, and scale real data

```

[3]: real_files = [
      Path(path) for path in sorted(glob.glob(str(PROJECT_ROOT / REAL_DATA_GLOB)))
    ]

```

```

real_ohlcvs = read_real_ohlcvs_csvs(real_files, EXPECTED_TICKERS)
check_calendar_coverage(real_ohlcvs, TRAIN_START, VALIDATION_END, "real_
↳train+validation")
check_calendar_coverage(real_ohlcvs, TEST_START, TEST_END, "real test")

periods, real_scaler = prepare_real_portfolio_periods(
    real_ohlcvs,
    EXPECTED_TICKERS,
    TRAIN_START,
    TRAIN_END,
    VALIDATION_START,
    VALIDATION_END,
    TEST_START,
    TEST_END,
    time_window=TIME_WINDOW,
    price_columns=PRICE_COLUMNS,
)
train = periods["train"]
validation = periods["validation"]
test = periods["test"]
ENV_KWARGS_BY_COMMISSION = {
    name: portfolio_environment_kwargs(
        commission,
        cwd=PROJECT_ROOT,
        initial_amount=INITIAL_AMOUNT,
        time_window=TIME_WINDOW,
        reward_scaling=REWARD_SCALING,
        price_columns=PRICE_COLUMNS,
    )
    for name, commission in COMMISSION_CONFIGS.items()
}

analysis_starts = {
    "train": pd.Timestamp(TRAIN_START),
    "validation": pd.Timestamp(VALIDATION_START),
    "test": pd.Timestamp(TEST_START),
}

period_summary = pd.DataFrame(
    [
        {
            "period": label,
            "input_first_date": frame["date"].min(),
            "analysis_first_date": analysis_starts[label],
            "last_date": frame["date"].max(),
            "input_dates": frame["date"].nunique(),
            "analysis_dates": frame.loc[
                frame["date"] >= analysis_starts[label], "date"
            ]
        }
    ]
)

```

```

        ].nunique(),
        "tickers": frame["tic"].nunique(),
    }
    for label, frame in periods.items()
]
)
print("per-ticker train scales:", real_scaler.scales)
period_summary

```

per-ticker train scales: {'AAPL': 235.1926606193721, 'AMZN': 201.1999969482422, 'GOOGL': 190.2827936474972, 'MSFT': 461.1034687451422, 'NVDA': 140.52837801062054}

```

[3]:      period input_first_date analysis_first_date  last_date  input_dates  \
0      train      2024-01-02      2024-01-01  2024-09-30          188
1  validation      2024-07-22      2024-10-01  2024-12-31          114
2      test      2024-10-21      2025-01-01  2025-12-31          300

      analysis_dates  tickers
0              188         5
1              64         5
2             250         5

```

1.3.1 Resolved reward-variant configuration

```

[4]: variant_configuration = pd.DataFrame(
    [
        {
            "variant": variant,
            "surprise_mode": resolved["surprise_mode"],
            "observation_dim": resolved["observation_dim"],
            "nominal_alpha": resolved["nominal_alpha"],
            "effective_alpha": resolved["effective_alpha"],
            "beta": resolved["beta"],
        }
        for variant in VARIANTS
        for resolved in [
            resolve_variant_reward_config(
                train,
                variant,
                ENV_KWARGS_BY_COMMISSION["with_fee"],
            )
        ]
    ]
)
variant_configuration.to_csv(
    ARTIFACT_ROOT / "reward_variant_configuration.csv", index=False
)

```

```
variant_configuration
```

```
[4]:
```

	variant	surprise_mode	observation_dim	nominal_alpha	\
0	baseline	none	756	0.00	
1	paper_faithful	paper_sum_nll	756	0.05	
2	robust_surprise	robust_centered_z	756	0.05	
3	surprise	legacy_positive_rms	756	0.05	
4	dejavu	none	756	0.00	
5	combined	legacy_positive_rms	756	0.05	

	effective_alpha	beta
0	0.000000	0.00
1	0.000066	0.00
2	0.050000	0.00
3	0.050000	0.00
4	0.000000	0.05
5	0.050000	0.05

1.3.2 Minimal real-control sanity check

```
[5]: sanity_env = PortfolioOptimizationGymnasiumEnv(
    train,
    **ENV_KWARGS_BY_COMMISSION["with_fee"],
)
sanity_observation, _ = sanity_env.reset(seed=SEEDS[0])
print("state:", sanity_observation["state"].shape)
print("last action:", sanity_observation["last_action"])
print("action space:", sanity_env.action_space)
print("max achievable single weight:", f"{sanity_env.max_achievable_weight():.6f}")
assert sanity_observation["state"].shape == (
    len(PRICE_COLUMNS),
    len(EXPECTED_TICKERS),
    TIME_WINDOW,
)
assert sanity_observation["last_action"].shape == (len(EXPECTED_TICKERS) + 1,)
assert sanity_env.action_space.shape == (len(EXPECTED_TICKERS) + 1,)
```

```
state: (3, 5, 50)
last action: [1. 0. 0. 0. 0. 0.]
action space: Box(-1.0, 1.0, (6,), float32)
max achievable single weight: 0.999773
```

1.4 3. Training process

The preflight fingerprints every semantic run before building a PPO environment. Complete cache hits load first; only misses train. The outer bar tracks runs and the inner bar tracks PPO timesteps. At timestep 0 and every EVAL_FREQ, deterministic external reward is measured on real

train and real validation. Only real-validation external reward selects the checkpoint. All runs use `with_fee=0.0025`.

```
[6]: run_specs = [
    {
        "variant": variant,
        "commission_name": commission_name,
        "commission": commission,
        "seed": seed,
    }
    for seed in SEEDS
    for commission_name, commission in COMMISSION_CONFIGS.items()
    for variant in VARIANTS
]
run_plans = [
    plan_portfolio_training(
        train=train,
        validation=validation,
        variant=spec["variant"],
        commission_name=spec["commission_name"],
        commission=spec["commission"],
        total_timesteps=TOTAL_TIMESTEPS,
        eval_freq=EVAL_FREQ,
        seed=spec["seed"],
        warmup_steps=WARMUP_STEPS,
        env_kwargs=ENV_KWARGS_BY_COMMISSION[spec["commission_name"]],
        ppo_kwargs=PPO_KWARGS,
        cache_dir=TRAIN_CACHE_ROOT,
        use_cache=USE_TRAIN_CACHE,
        force_retrain=FORCE_RETRAIN,
        replay_capacity=REPLAY_CAPACITY,
    )
    for spec in tqdm(run_specs, desc="Checking training cache", unit="run")
]
cache_hit_plans = [plan for plan in run_plans if plan.cache_hit]
cache_miss_plans = [plan for plan in run_plans if not plan.cache_hit]
print(
    "cache preflight:",
    f"{len(cache_hit_plans)} hit(s), {len(cache_miss_plans)} miss(es)",
)

def train_from_plan(plan):
    return train_portfolio_variant(
        train=train,
        validation=validation,
        plan=plan,
```

```

        total_timesteps=TOTAL_TIMESTEPS,
        eval_freq=EVAL_FREQ,
        warmup_steps=WARMUP_STEPS,
        env_kwargs=ENV_KWARGS_BY_COMMISSION[plan.commission_name],
        ppo_kwargs=PPO_KWARGS,
        show_progress=True,
        replay_capacity=REPLAY_CAPACITY,
    )

runs_by_key = {}
for plan in tqdm(cache_hit_plans, desc="Loading cache hits", unit="run"):
    key = (plan.variant, plan.commission_name, plan.seed)
    runs_by_key[key] = train_from_plan(plan)

if cache_miss_plans:
    miss_progress = tqdm(
        cache_miss_plans,
        desc="Training cache misses",
        unit="run",
        position=0,
    )
    for plan in miss_progress:
        miss_progress.set_postfix_str(
            f"{plan.variant} | {plan.commission_name} | seed {plan.seed}",
            refresh=True,
        )
        key = (plan.variant, plan.commission_name, plan.seed)
        runs_by_key[key] = train_from_plan(plan)
else:
    print("No cache misses to train.")

runs = [
    runs_by_key[(plan.variant, plan.commission_name, plan.seed)]
    for plan in run_plans
]
run_table = pd.DataFrame(
    [
        {
            "variant": run.variant,
            "commission": run.commission_name,
            "seed": run.seed,
            "cache_hit": run.cache_hit,
            "cache_id": run.cache_key[:12],
            "best_timestep": run.best_timestep,
            "best_real_validation_reward": (
                run.validation_log["real_valid_reward"].max()
            )
        }
    ]
)

```

```

        if not run.validation_log.empty
        else np.nan
    ),
}
for run in runs
]
)
run_table.to_csv(ARTIFACT_ROOT / "training_runs.csv", index=False)
print("training cache hits:", f"{int(run_table['cache_hit'].sum())}/{len(run_table)}")
run_table

```

Checking training cache: 100%| | 18/18 [00:00<00:00, 385.11run/s]

cache preflight: 13 hit(s), 5 miss(es)

Loading cache hits: 0%| | 0/13 [00:00<?,
?run/s]/opt/anaconda3/envs/finrl_exp/lib/python3.10/site-
packages/torch/cuda/_init__.py:188: UserWarning: CUDA initialization: The
NVIDIA driver on your system is too old (found version 12080). Please update
your GPU driver by downloading and installing a new version from the URL:
<http://www.nvidia.com/Download/index.aspx> Alternatively, go to:
<https://pytorch.org> to install a PyTorch version that has been compiled with
your version of the CUDA driver. (Triggered internally at
/_w/pytorch/pytorch/c10/cuda/CUDAFunctions.cpp:119.)

return torch._C._cuda_getDeviceCount() > 0

Loading cache hits: 100%| | 13/13 [00:01<00:00, 12.82run/s]

Training cache misses: 100%| | 5/5 [2:42:39<00:00, 1951.97s/run,
combined | with_fee | seed 2]

training cache hits: 13/18

[6]:	variant	commission	seed	cache_hit	cache_id	best_timestep \
0	baseline	with_fee	0	True	08616c161c05	180000
1	paper_faithful	with_fee	0	True	a40c1f5d3e98	160000
2	robust_surprise	with_fee	0	True	f7c50fef64f4	180000
3	surprise	with_fee	0	True	6e83c01b87b4	180000
4	dejavu	with_fee	0	True	088e6ed523b1	180000
5	combined	with_fee	0	True	a6dfe88f9046	180000
6	baseline	with_fee	1	True	cf3deae2051d	0
7	paper_faithful	with_fee	1	True	8197731af428	140000
8	robust_surprise	with_fee	1	True	0db901d5d60f	100000
9	surprise	with_fee	1	True	c22f8ecec18b	0
10	dejavu	with_fee	1	True	42b8b06d7c0f	80000
11	combined	with_fee	1	True	530cac9f92e8	60000
12	baseline	with_fee	2	True	85fce99aa7f0	60000
13	paper_faithful	with_fee	2	False	85e752acd645	20000

14	robust_surprise	with_fee	2	False	3a92ec4693dc	60000
15	surprise	with_fee	2	False	53eed677c2cb	60000
16	dejavu	with_fee	2	False	49bf645ff657	0
17	combined	with_fee	2	False	190e29b420d5	20000

	best_real_validation_reward
0	0.166008
1	0.143015
2	0.130481
3	0.166008
4	0.164494
5	0.156910
6	0.122985
7	0.132450
8	0.124422
9	0.122985
10	0.123184
11	0.123784
12	0.135713
13	0.123609
14	0.126090
15	0.135713
16	0.122723
17	0.125431

1.5 4. Learning curves

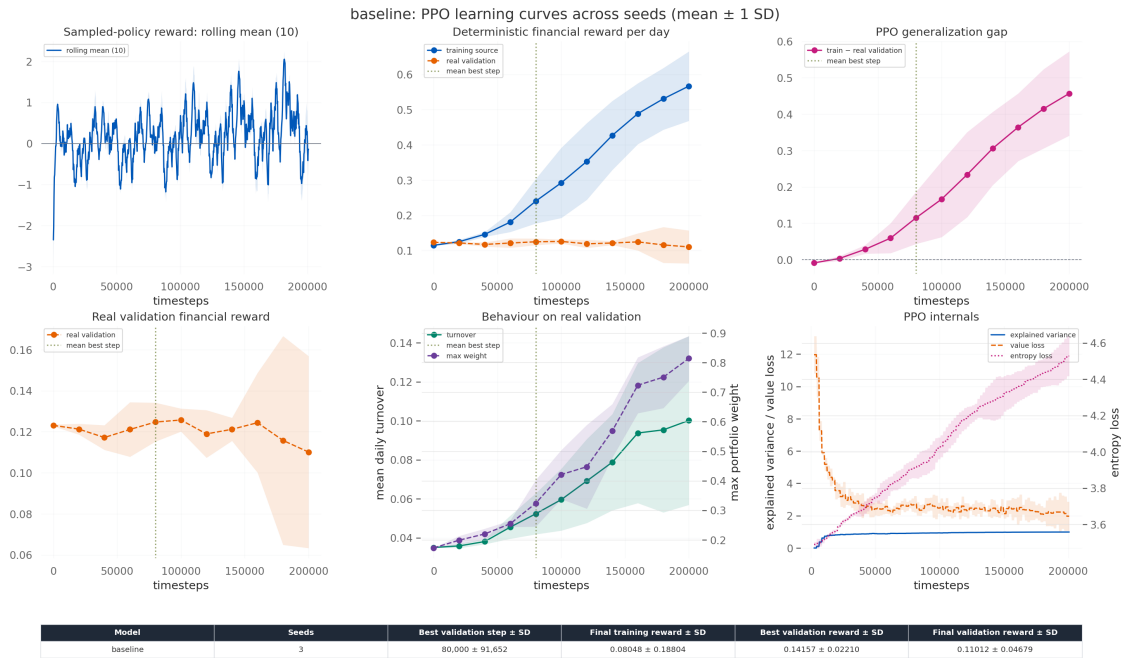
Each figure contains one reward variant aggregated across the same seeds as the original experiment. Bands show ± 1 SD. Deterministic train and validation curves always use external financial reward; sampled-policy reward is the reward PPO actually received during training. All figures use the 0.25% fee condition.

```
[7]: learning_paths = {}
for commission_name in COMMISSION_CONFIGS:
    commission_runs = [
        run for run in runs if run.commission_name == commission_name
    ]
    figures = plot_learning_curves_by_variant(commission_runs)
    for variant, figure in figures.items():
        path = ARTIFACT_ROOT / (
            f"{commission_name}_{variant}_learning_curves_all_seeds.png"
        )
        figure.savefig(path, dpi=160, bbox_inches="tight")
        plt.close(figure)
        learning_paths[(commission_name, variant)] = path
learning_paths
```

```
[7]: {('with_fee',
      'baseline'): PosixPath('/home/cwei1213/FinRL_Exp/results/ppo_portfolio_intrinsic_reward_ablation/full/with_fee__baseline__learning_curves_all_seeds.png'),
      ('with_fee',
      'paper_faithful'): PosixPath('/home/cwei1213/FinRL_Exp/results/ppo_portfolio_intrinsic_reward_ablation/full/with_fee__paper_faithful__learning_curves_all_seeds.png'),
      ('with_fee',
      'robust_surprise'): PosixPath('/home/cwei1213/FinRL_Exp/results/ppo_portfolio_intrinsic_reward_ablation/full/with_fee__robust_surprise__learning_curves_all_seeds.png'),
      ('with_fee',
      'surprise'): PosixPath('/home/cwei1213/FinRL_Exp/results/ppo_portfolio_intrinsic_reward_ablation/full/with_fee__surprise__learning_curves_all_seeds.png'),
      ('with_fee',
      'dejavu'): PosixPath('/home/cwei1213/FinRL_Exp/results/ppo_portfolio_intrinsic_reward_ablation/full/with_fee__dejavu__learning_curves_all_seeds.png'),
      ('with_fee',
      'combined'): PosixPath('/home/cwei1213/FinRL_Exp/results/ppo_portfolio_intrinsic_reward_ablation/full/with_fee__combined__learning_curves_all_seeds.png')}
```

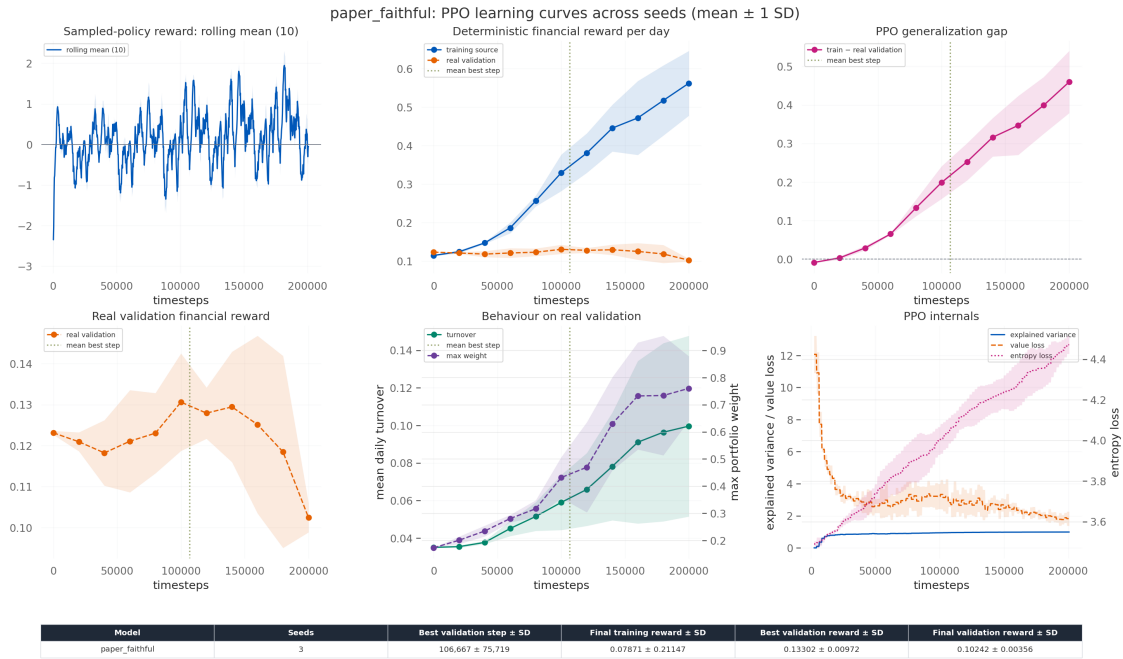
```
[8]: NotebookImage(filename=str(learning_paths[("with_fee", "baseline")]))
```

[8]:



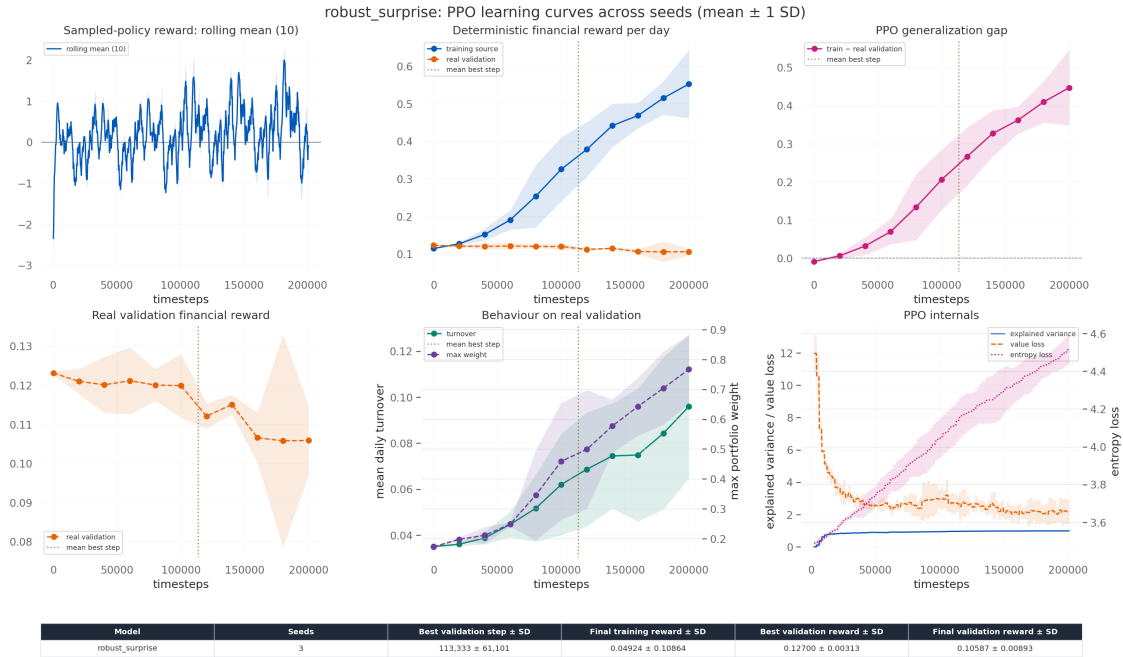
```
[9]: NotebookImage(filename=str(learning_paths[("with_fee", "paper_faithful")]))
```

[9]:



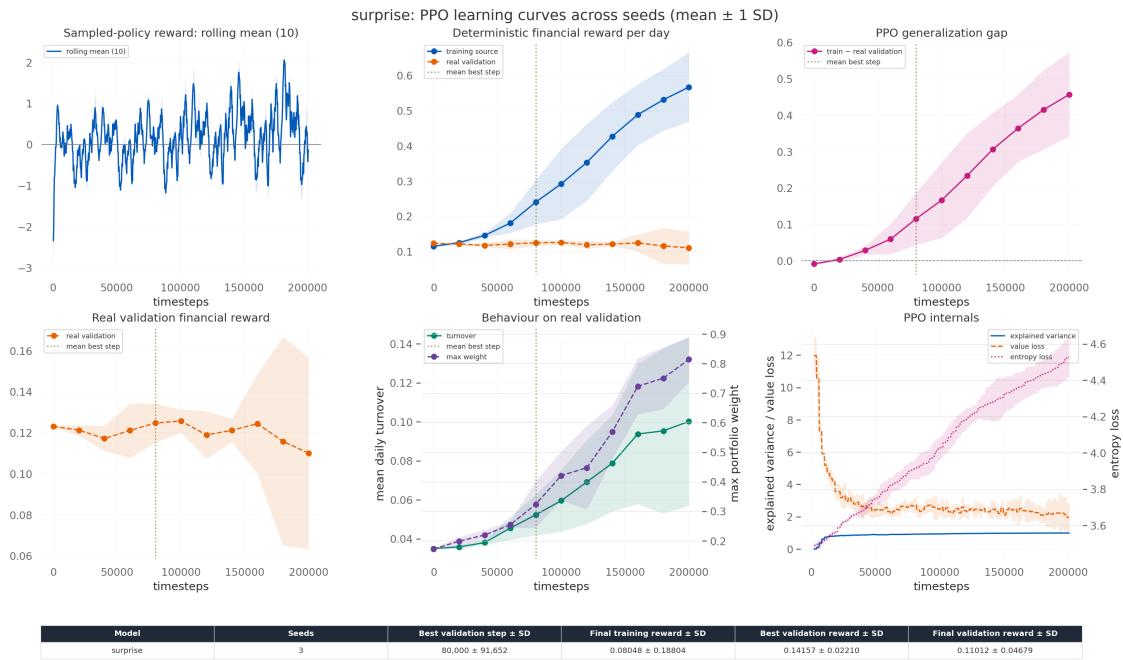
[10]: NotebookImage(filename=str(learning_paths[("with_fee", "robust_surprise"))])

[10]:



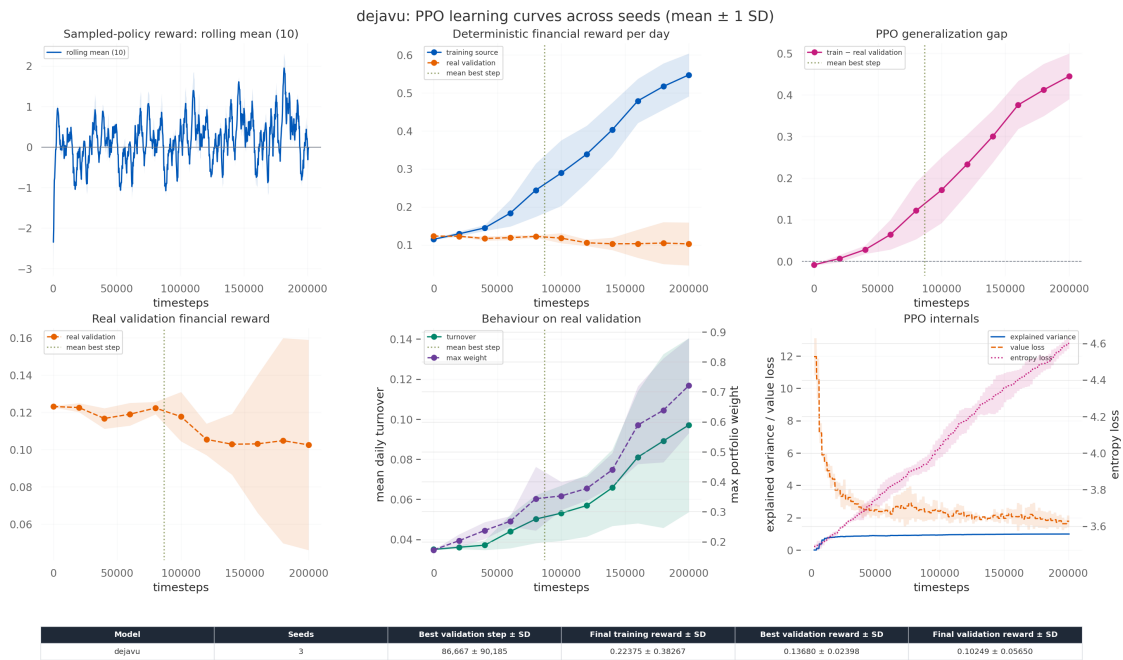
```
[11]: NotebookImage(filename=str(learning_paths[("with_fee", "surprise"))])
```

```
[11]:
```



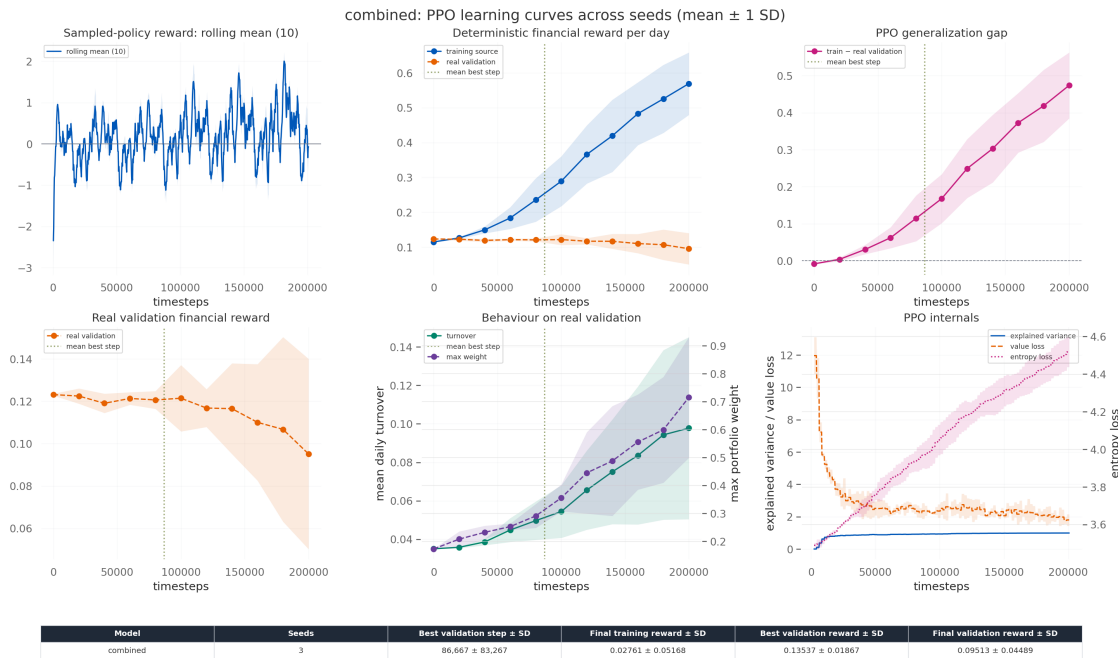
```
[12]: NotebookImage(filename=str(learning_paths[("with_fee", "dejavu"))])
```

```
[12]:
```



```
[13]: NotebookImage(filename=str(learning_paths[("with_fee", "combined"))])
```

```
[13]:
```



1.5.1 Intrinsic-reward diagnostics (additional)

These panels compare the paper-faithful summed NLL, robust centered-z score, legacy positive-RMS score, Déjà vu, and combined reward. A large or negative paper score is an experimental result rather than a plotting error; its effective α is dimension adjusted before reward composition.

```
[14]: intrinsic_log_frames = []
for run in runs:
    if run.variant == "baseline" or run.intrinsic_log.empty:
        continue
    frame = run.intrinsic_log.copy()
    frame["variant"] = run.variant
    frame["commission_name"] = run.commission_name
    frame["seed"] = run.seed
    intrinsic_log_frames.append(frame)
intrinsic_log = pd.concat(intrinsic_log_frames, ignore_index=True)

intrinsic_paths = {}
intrinsic_summaries = []
for commission_name in COMMISSION_CONFIGS:
    commission_log = intrinsic_log.loc[
        intrinsic_log["commission_name"] == commission_name
    ]
```



```

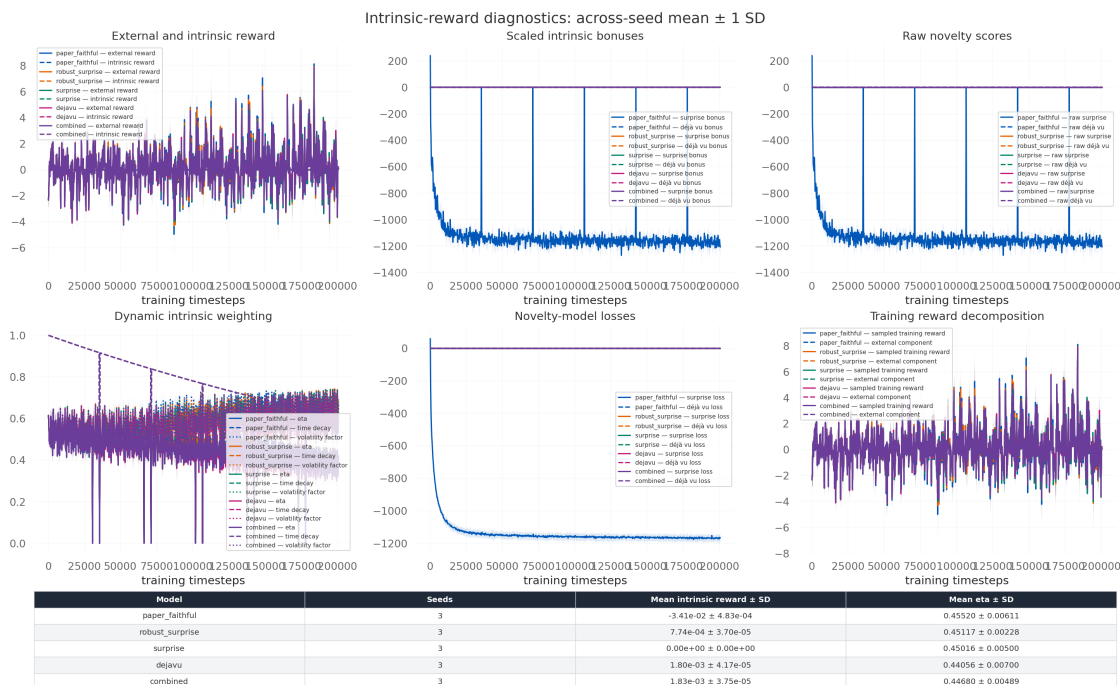
figure, curve_summary, model_summary = plot_intrinsic_diagnostics(
    commission_log
)
path = ARTIFACT_ROOT / f"{commission_name}__intrinsic_reward_diagnostics.
↳png"
figure.savefig(path, dpi=160, bbox_inches="tight")
plt.close(figure)
intrinsic_paths[commission_name] = path
model_summary.insert(1, "commission_name", commission_name)
intrinsic_summaries.append(model_summary)
pd.concat(intrinsic_summaries, ignore_index=True).to_csv(
    ARTIFACT_ROOT / "intrinsic_reward_summary.csv", index=False
)
intrinsic_paths

```

[14]: {'with_fee': PosixPath('/home/cwei1213/FinRL_Exp/results/ppo_portfolio_intrinsic_reward_ablation/full/with_fee__intrinsic_reward_diagnostics.png')}

[15]: NotebookImage(filename=str(intrinsic_paths["with_fee"]))

[15]:



1.6 5. External-reward backtests and uniform benchmark

Every saved PPO checkpoint is evaluated deterministically on train, validation, and test using the unmodified portfolio environment with a 0.25% fee. The benchmark uses the same uniform portfolio action as the Real-vs-Synthetic notebook. No intrinsic reward or novelty-model update

occurs here.

```
[16]: (
    ppo_period_summary,
    period_results,
    benchmark_summary,
    benchmark_results,
) = evaluate_portfolio_periods(
    periods,
    runs,
    cwd=PROJECT_ROOT,
    initial_amount=INITIAL_AMOUNT,
    time_window=TIME_WINDOW,
    reward_scaling=REWARD_SCALING,
)
backtest_summary = pd.concat(
    [ppo_period_summary, benchmark_summary], ignore_index=True
)
ppo_period_summary.to_csv(ARTIFACT_ROOT / "ppo_period_metrics.csv", index=False)
benchmark_summary.to_csv(ARTIFACT_ROOT / "benchmark_metrics.csv", index=False)
backtest_summary.to_csv(ARTIFACT_ROOT / "backtest_results.csv", index=False)
backtest_summary
```

```
[16]:
```

	period	commission_name	commission	variant	seed \
0	train	with_fee	0.0025	baseline	0
1	train	with_fee	0.0025	paper_faithful	0
2	train	with_fee	0.0025	robust_surprise	0
3	train	with_fee	0.0025	surprise	0
4	train	with_fee	0.0025	dejavu	0
5	train	with_fee	0.0025	combined	0
6	train	with_fee	0.0025	baseline	1
7	train	with_fee	0.0025	paper_faithful	1
8	train	with_fee	0.0025	robust_surprise	1
9	train	with_fee	0.0025	surprise	1
10	train	with_fee	0.0025	dejavu	1
11	train	with_fee	0.0025	combined	1
12	train	with_fee	0.0025	baseline	2
13	train	with_fee	0.0025	paper_faithful	2
14	train	with_fee	0.0025	robust_surprise	2
15	train	with_fee	0.0025	surprise	2
16	train	with_fee	0.0025	dejavu	2
17	train	with_fee	0.0025	combined	2
18	validation	with_fee	0.0025	baseline	0
19	validation	with_fee	0.0025	paper_faithful	0
20	validation	with_fee	0.0025	robust_surprise	0
21	validation	with_fee	0.0025	surprise	0
22	validation	with_fee	0.0025	dejavu	0

23	validation	with_fee	0.0025	combined	0
24	validation	with_fee	0.0025	baseline	1
25	validation	with_fee	0.0025	paper_faithful	1
26	validation	with_fee	0.0025	robust_surprise	1
27	validation	with_fee	0.0025	surprise	1
28	validation	with_fee	0.0025	dejavu	1
29	validation	with_fee	0.0025	combined	1
30	validation	with_fee	0.0025	baseline	2
31	validation	with_fee	0.0025	paper_faithful	2
32	validation	with_fee	0.0025	robust_surprise	2
33	validation	with_fee	0.0025	surprise	2
34	validation	with_fee	0.0025	dejavu	2
35	validation	with_fee	0.0025	combined	2
36	test	with_fee	0.0025	baseline	0
37	test	with_fee	0.0025	paper_faithful	0
38	test	with_fee	0.0025	robust_surprise	0
39	test	with_fee	0.0025	surprise	0
40	test	with_fee	0.0025	dejavu	0
41	test	with_fee	0.0025	combined	0
42	test	with_fee	0.0025	baseline	1
43	test	with_fee	0.0025	paper_faithful	1
44	test	with_fee	0.0025	robust_surprise	1
45	test	with_fee	0.0025	surprise	1
46	test	with_fee	0.0025	dejavu	1
47	test	with_fee	0.0025	combined	1
48	test	with_fee	0.0025	baseline	2
49	test	with_fee	0.0025	paper_faithful	2
50	test	with_fee	0.0025	robust_surprise	2
51	test	with_fee	0.0025	surprise	2
52	test	with_fee	0.0025	dejavu	2
53	test	with_fee	0.0025	combined	2
54	train	with_fee	0.0025	buy_and_hold	<NA>
55	validation	with_fee	0.0025	buy_and_hold	<NA>
56	test	with_fee	0.0025	buy_and_hold	<NA>

	best_timestep	final_value	cumulative_return	cagr	sharpe	\
0	180000.0	218738.984375	1.187390	3.175753	4.577910	
1	160000.0	204020.093750	1.040201	2.676967	3.878805	
2	180000.0	211875.687500	1.118757	2.939603	4.385522	
3	180000.0	218738.984375	1.187390	3.175753	4.577910	
4	180000.0	218475.328125	1.184753	3.166566	4.566208	
5	180000.0	215984.593750	1.159846	3.080234	4.650239	
6	0.0	116981.390625	0.169814	0.331641	1.560930	
7	140000.0	197445.187500	0.974452	2.463467	4.186952	
8	100000.0	177910.796875	0.779108	1.863460	3.476430	
9	0.0	116981.390625	0.169814	0.331641	1.560930	
10	80000.0	156183.140625	0.561831	1.257318	3.308180	

11	60000.0	134277.250000	0.342772	0.712946	2.718140
12	60000.0	123272.976562	0.232730	0.465321	1.964820
13	20000.0	118941.882812	0.189419	0.372676	1.704185
14	60000.0	125153.656250	0.251537	0.506400	2.071610
15	60000.0	123272.976562	0.232730	0.465321	1.964820
16	0.0	117355.054688	0.173551	0.339419	1.576806
17	20000.0	119314.679688	0.193147	0.380542	1.648645
18	180000.0	111209.468750	0.112095	0.519439	2.386250
19	160000.0	109584.890625	0.095849	0.433898	2.100671
20	180000.0	108709.359375	0.087094	0.389316	2.171253
21	180000.0	111209.468750	0.112095	0.519439	2.386250
22	180000.0	111101.750000	0.111017	0.513652	2.663303
23	180000.0	110563.750000	0.105638	0.484996	2.662498
24	0.0	108189.117188	0.081891	0.363320	1.963038
25	140000.0	108846.453125	0.088465	0.396228	2.068382
26	100000.0	108288.625000	0.082886	0.368264	1.933808
27	0.0	108189.117188	0.081891	0.363320	1.963038
28	80000.0	108202.906250	0.082029	0.364004	2.091822
29	60000.0	108244.421875	0.082444	0.366066	2.069684
30	60000.0	109073.968750	0.090740	0.407754	2.112540
31	20000.0	108232.296875	0.082323	0.365464	2.032246
32	60000.0	108404.296875	0.084043	0.374028	2.072272
33	60000.0	109073.968750	0.090740	0.407754	2.112540
34	0.0	108170.921875	0.081709	0.362418	1.958585
35	20000.0	108358.562500	0.083586	0.371747	1.987825
36	180000.0	113825.937500	0.138259	0.139439	0.538988
37	160000.0	114037.781250	0.140378	0.141577	0.550909
38	180000.0	116631.734375	0.166317	0.167754	0.636566
39	180000.0	113825.937500	0.138259	0.139439	0.538988
40	180000.0	115189.937500	0.151899	0.153203	0.586473
41	180000.0	118906.804688	0.189068	0.190716	0.709417
42	0.0	122735.789062	0.227358	0.229371	1.017584
43	140000.0	115486.921875	0.154869	0.156200	0.593604
44	100000.0	117320.968750	0.173210	0.174710	0.634595
45	0.0	122735.789062	0.227358	0.229371	1.017584
46	80000.0	120323.937500	0.203239	0.205022	0.798051
47	60000.0	122769.742188	0.227697	0.229714	0.975754
48	60000.0	122609.414062	0.226094	0.228095	0.957434
49	20000.0	121875.359375	0.218754	0.220684	0.982390
50	60000.0	122569.046875	0.225690	0.227688	0.971301
51	60000.0	122609.414062	0.226094	0.228095	0.957434
52	0.0	123281.203125	0.232812	0.234878	1.032120
53	20000.0	123250.710938	0.232507	0.234570	1.001577
54	NaN	120467.851562	0.204679	0.405005	1.565382
55	NaN	109816.359375	0.098164	0.445861	1.970268
56	NaN	127493.843750	0.274938	0.277418	1.030812

	annualized_volatility	max_drawdown	turnover	reward_per_day
0	0.324317	-0.125620	0.120566	0.567180
1	0.352195	-0.139123	0.097452	0.516702
2	0.325259	-0.120730	0.106940	0.544079
3	0.324317	-0.125620	0.120566	0.567180
4	0.324690	-0.118827	0.118078	0.566306
5	0.313539	-0.113048	0.116598	0.557998
6	0.195826	-0.146432	0.022516	0.113656
7	0.308543	-0.135105	0.096025	0.492964
8	0.317478	-0.151062	0.077139	0.417473
9	0.195826	-0.146432	0.022516	0.113656
10	0.256301	-0.135521	0.057469	0.323086
11	0.205967	-0.137580	0.040532	0.213577
12	0.205277	-0.149883	0.028318	0.151617
13	0.197369	-0.144146	0.023496	0.125699
14	0.208363	-0.146982	0.029878	0.162588
15	0.205277	-0.149883	0.028318	0.151617
16	0.197797	-0.146794	0.022578	0.115967
17	0.208903	-0.150927	0.024462	0.127967
18	0.182293	-0.043824	0.132921	0.166008
19	0.179219	-0.039582	0.118502	0.143015
20	0.157121	-0.037207	0.115830	0.130481
21	0.182293	-0.043824	0.132921	0.166008
22	0.160529	-0.036923	0.137107	0.164494
23	0.152951	-0.035192	0.135380	0.156910
24	0.164812	-0.033793	0.034874	0.122985
25	0.168221	-0.039952	0.099241	0.132450
26	0.169577	-0.039608	0.082892	0.124422
27	0.164812	-0.033793	0.034874	0.122985
28	0.154096	-0.033869	0.062143	0.123184
29	0.156668	-0.033799	0.050525	0.123784
30	0.168650	-0.035029	0.038889	0.135713
31	0.159559	-0.031182	0.034953	0.123609
32	0.159494	-0.032048	0.038637	0.126090
33	0.168650	-0.035029	0.038889	0.135713
34	0.164851	-0.033632	0.035120	0.122723
35	0.165953	-0.032403	0.036053	0.125431
36	0.373400	-0.310440	0.100920	0.051800
37	0.360420	-0.291633	0.080396	0.052544
38	0.331911	-0.257709	0.090520	0.061540
39	0.373400	-0.310440	0.100920	0.051800
40	0.348291	-0.302771	0.105153	0.056565
41	0.319286	-0.255671	0.103184	0.069268
42	0.228220	-0.223232	0.016643	0.081946
43	0.347202	-0.296612	0.074864	0.057595
44	0.351560	-0.295038	0.055851	0.063897
45	0.228220	-0.223232	0.016643	0.081946

46	0.283960	-0.253700	0.043801	0.074007
47	0.241510	-0.224787	0.031370	0.082056
48	0.245767	-0.231302	0.021150	0.081533
49	0.229446	-0.223619	0.017147	0.079131
50	0.240660	-0.224398	0.022108	0.081402
51	0.245767	-0.231302	0.021150	0.081533
52	0.229648	-0.224858	0.016741	0.083719
53	0.238428	-0.227888	0.017878	0.083620
54	0.234943	-0.173161	0.025397	0.134937
55	0.197009	-0.040570	0.040155	0.146311
56	0.273230	-0.262938	0.018132	0.097159

1.7 6. Backtest Sharpe and test metrics

The Sharpe plot retains every seed and marks the seed nearest each variant's median test Sharpe. The metric plot shows all-seed mean $\pm$ SD and the deterministic uniform benchmark, all under the 0.25% fee condition.

```
[17]: sharpe_paths = {}
metric_paths = {}
test_model_summaries = []
for commission_name in COMMISSION_CONFIGS:
    test_summary = ppo_period_summary.loc[
        (ppo_period_summary["period"] == "test")
        & (ppo_period_summary["commission_name"] == commission_name)
    ].reset_index(drop=True)
    test_benchmark = benchmark_summary.loc[
        (benchmark_summary["period"] == "test")
        & (benchmark_summary["commission_name"] == commission_name)
    ].iloc[0]

    sharpe_figure, representatives = plot_sharpe_seed_selection(test_summary)
    sharpe_path = ARTIFACT_ROOT / \
    f"{commission_name}__median_sharpe_seed_selection.png"
    sharpe_figure.savefig(sharpe_path, dpi=160, bbox_inches="tight")
    plt.close(sharpe_figure)
    representatives.to_csv(
        ARTIFACT_ROOT / f"{commission_name}__median_sharpe_representatives.csv",
        index=False,
    )
    sharpe_paths[commission_name] = sharpe_path

    metrics_figure, model_summary = plot_test_metric_comparison(
        test_summary, test_benchmark
    )
    metrics_path = ARTIFACT_ROOT / \
    f"{commission_name}__test_metrics_vs_benchmark.png"
```

```

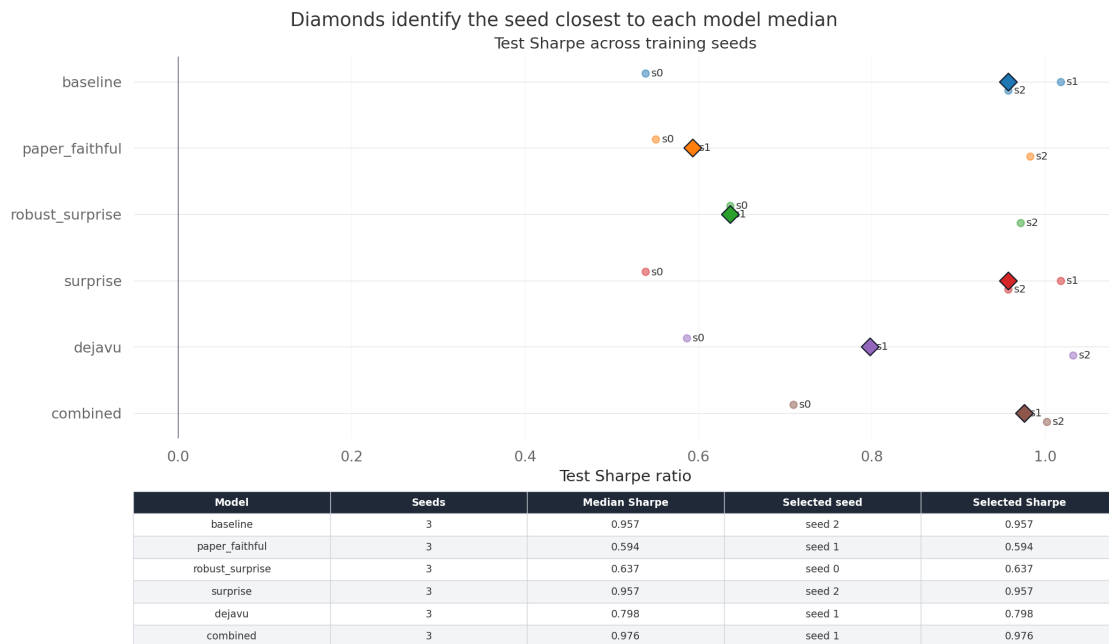
metrics_figure.savefig(metrics_path, dpi=160, bbox_inches="tight")
plt.close(metrics_figure)
model_summary.insert(1, "commission_name", commission_name)
test_model_summaries.append(model_summary)
metric_paths[commission_name] = metrics_path

test_model_summary = pd.concat(test_model_summaries, ignore_index=True)
test_model_summary.to_csv(
    ARTIFACT_ROOT / "test_metrics_aggregated.csv", index=False
)

```

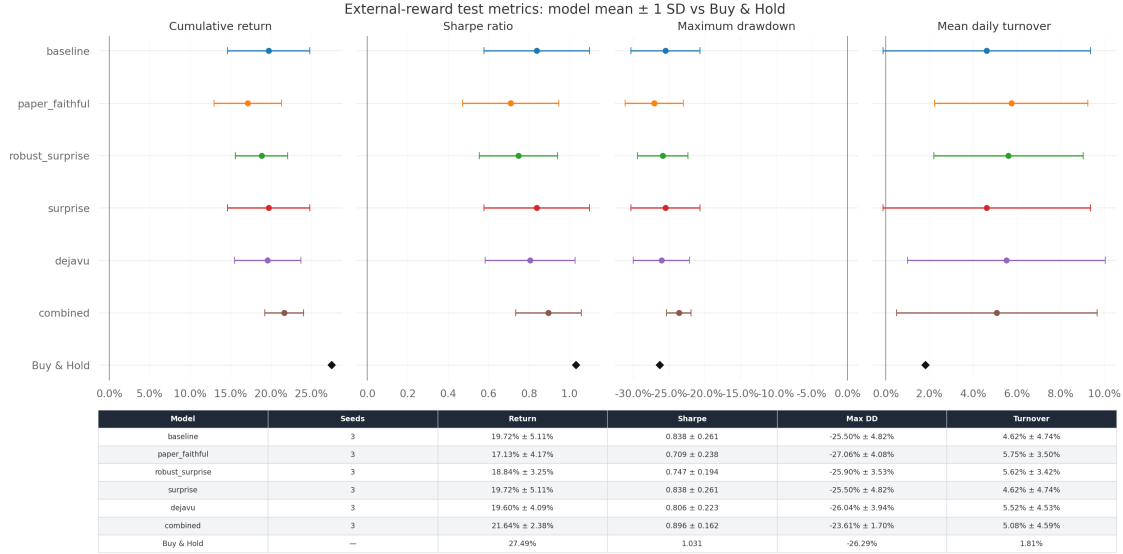
```
[18]: NotebookImage(filename=str(sharpe_paths["with_fee"]))
```

[18]:



```
[19]: NotebookImage(filename=str(metric_paths["with_fee"]))
```

[19]:



1.8 7. Train, validation, and test equity curves

All plots use Growth of \$1 under `with_fee=0.0025`. Reward variants are aggregated across seeds as mean $\pm$ SD; the uniform benchmark is the dashed black line. Tables report the exact ranking for the corresponding period.

```
[20]: equity_paths = {}
period_titles = {
    "train": f"Real train [{TRAIN_START}, {TRAIN_END}]",
    "validation": f"Real validation [{VALIDATION_START}, {VALIDATION_END}]",
    "test": f"Real test [{TEST_START}, {TEST_END}]",
}
for commission_name in COMMISSION_CONFIGS:
    for period in ("train", "validation", "test"):
        figure, equity, ranking = plot_period_equity_comparison(
            period,
            period_results[commission_name][period],
            benchmark_results[commission_name][period],
            f"{period_titles[period]} - {commission_name}",
        )
        path = ARTIFACT_ROOT / (
            f"{commission_name}__ppo_vs_benchmark_{period}_equity_curves.png"
        )
        figure.savefig(path, dpi=160, bbox_inches="tight")
        plt.close(figure)
        equity.to_csv(
            ARTIFACT_ROOT / f"{commission_name}__{period}_equity_curves.csv",
            index=False,
```



```

    )
    ranking.to_csv(
        ARTIFACT_ROOT / f"{commission_name}__{period}_equity_ranking.csv",
        index=False,
    )
    equity_paths[(commission_name, period)] = path
equity_paths

```

```

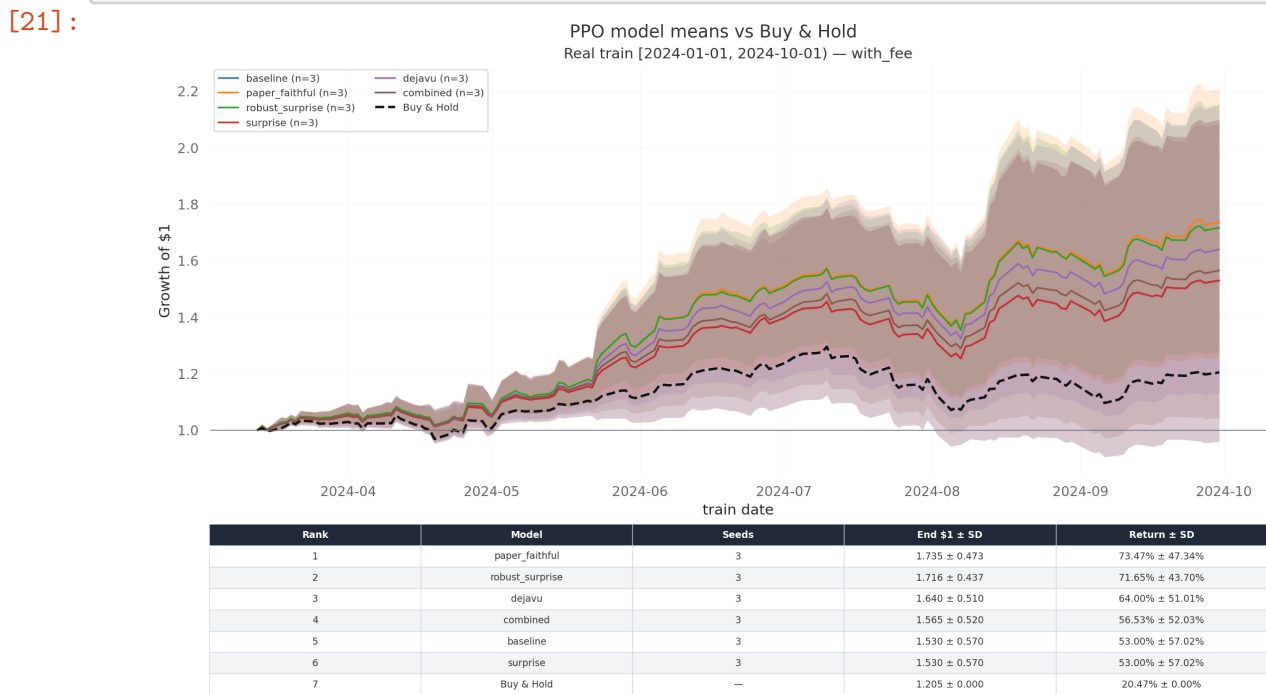
[20]: {('with_fee',
        'train'): PosixPath('/home/cwei1213/FinRL_Exp/results/ppo_portfolio_intrinsic_reward_ablation/full/with_fee_ppo_vs_benchmark_train_equity_curves.png'),
        ('with_fee',
         'validation'): PosixPath('/home/cwei1213/FinRL_Exp/results/ppo_portfolio_intrinsic_reward_ablation/full/with_fee_ppo_vs_benchmark_validation_equity_curves.png'),
        ('with_fee',
         'test'): PosixPath('/home/cwei1213/FinRL_Exp/results/ppo_portfolio_intrinsic_reward_ablation/full/with_fee_ppo_vs_benchmark_test_equity_curves.png')}

```

```

[21]: NotebookImage(filename=str(equity_paths[("with_fee", "train")]))

```

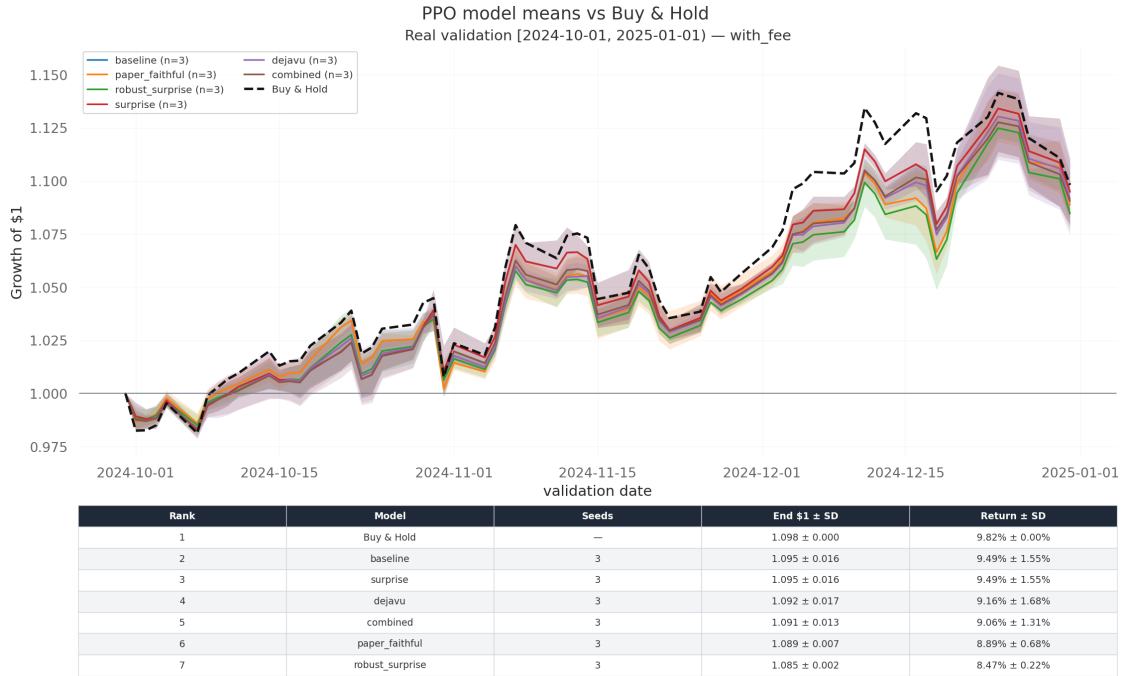


```

[22]: NotebookImage(filename=str(equity_paths[("with_fee", "validation")]))

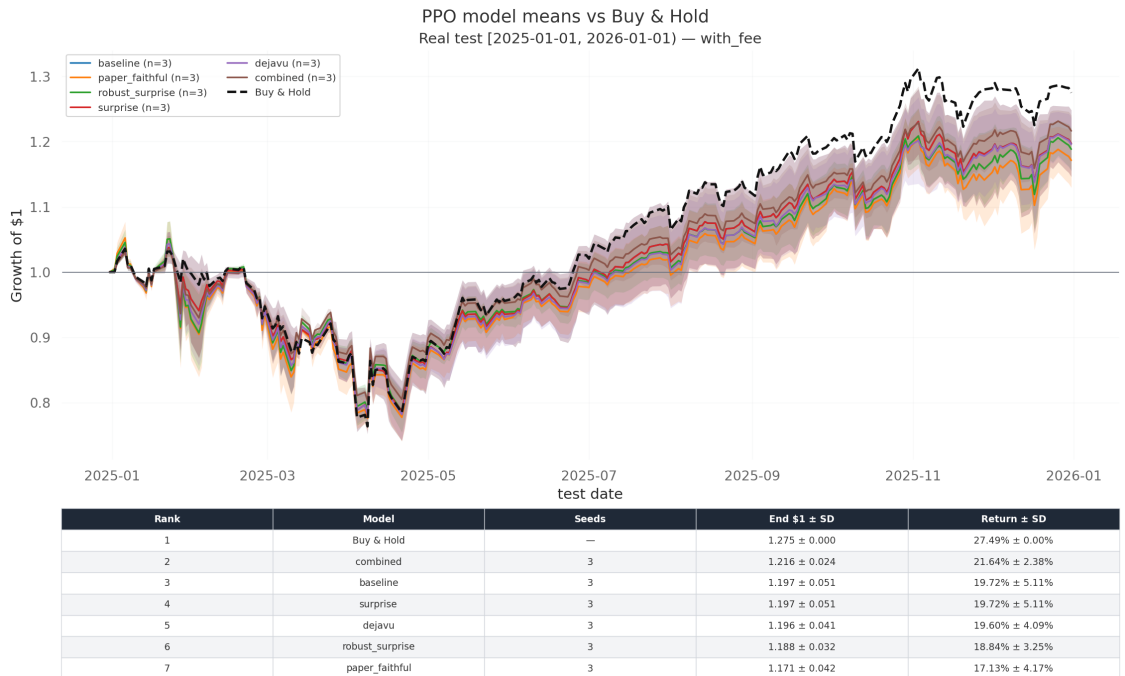
```

[22]:



[23]: NotebookImage(filename=str(equity_paths[("with_fee", "test")]))

[23]:



1.9 8. Portfolio weights

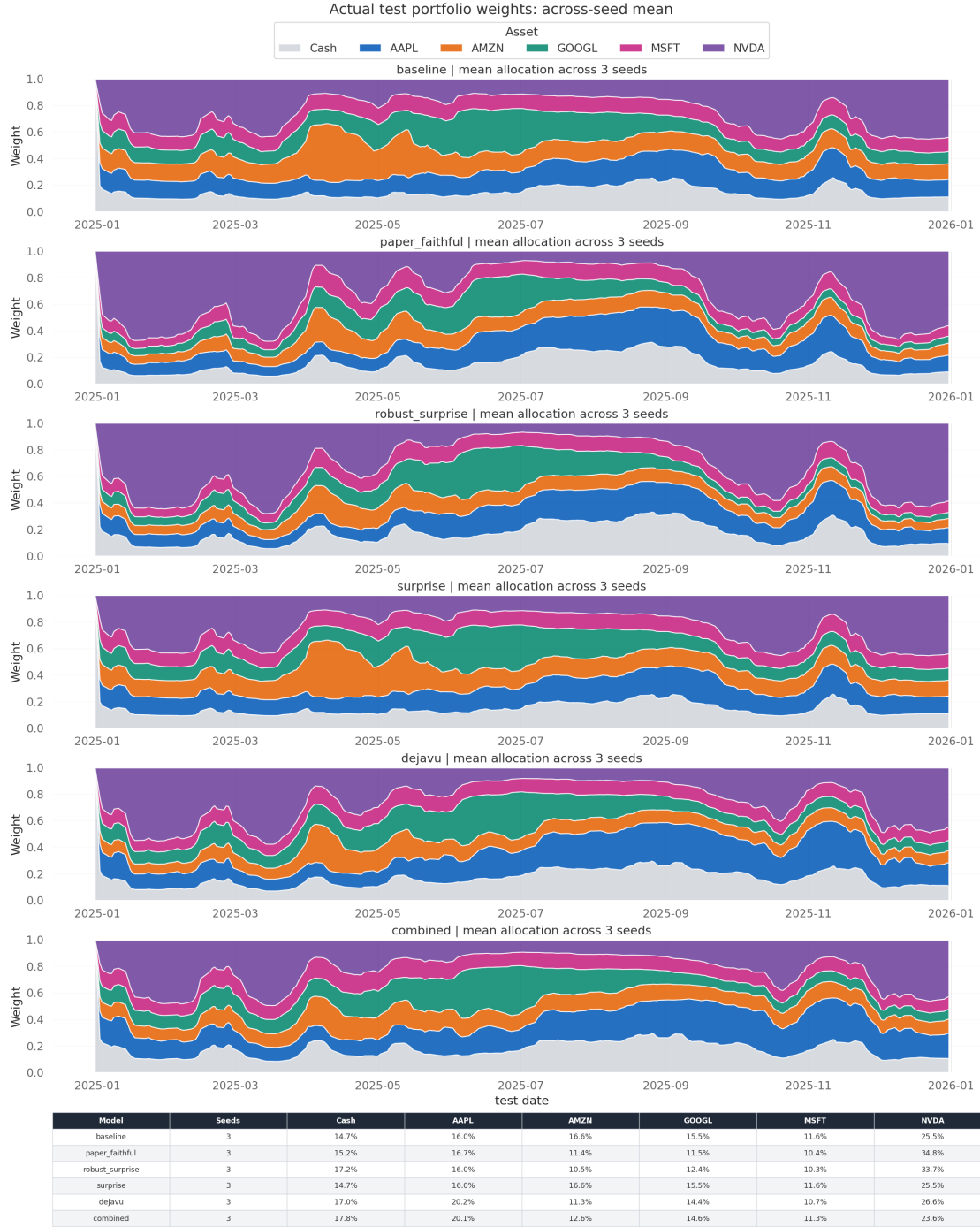
The stacked areas show chosen cash and stock weights through the real test period, averaged across seeds for each reward variant under the 0.25% fee condition. The table reports exact mean weights.

```
[24]: weight_paths = {}
weight_summaries = []
for commission_name in COMMISSION_CONFIGS:
    figure, mean_weights = plot_test_portfolio_weights(
        period_results[commission_name]["test"]
    )
    path = ARTIFACT_ROOT / f"{commission_name}__real_test_portfolio_weights.png"
    figure.savefig(path, dpi=160, bbox_inches="tight")
    plt.close(figure)
    mean_weights.insert(1, "commission_name", commission_name)
    weight_summaries.append(mean_weights)
    weight_paths[commission_name] = path
pd.concat(weight_summaries, ignore_index=True).to_csv(
    ARTIFACT_ROOT / "mean_test_portfolio_weights.csv", index=False
)
weight_paths
```

```
[24]: {'with_fee': PosixPath('/home/cwei1213/FinRL_Exp/results/ppo_portfolio_intrinsic_reward_ablation/full/with_fee__real_test_portfolio_weights.png')}
```

```
[25]: NotebookImage(filename=str(weight_paths["with_fee"]))
```

```
[25]:
```



1.10 9. Interpretation checklist

- Compare all variants under the single `with_fee=0.0025` condition.
- Treat **surprise** as the current/legacy implementation, not the robust default.
- Paper-faithful negative rewards or scale spikes are valid stability findings.
- Compare differences with cross-seed bands, not only point estimates.

- Intrinsic diagnostics explain the mechanism; they are not financial P&L.
- Backtest, Sharpe, equity, and portfolio-weight figures use external outcomes only.
- Baseline uses the same real-data PortfolioOptimization + EIIE PPO control; only the training reward differs.

1.11 Next steps

Run smoke mode first, then the formal 18-run experiment. The complete intrinsic controller checkpoint includes its replay pool and can be large. The paper-faithful variant should always receive a smoke check before a long run because summed Gaussian NLL may be negative or exhibit large spikes.

```
FINRL_INTRINSIC_RUN_MODE=smoke \
FINRL_USE_TRAIN_CACHE=1 \
FINRL_DEVICE=cpu \
jupyter nbconvert --to notebook --execute --inplace \
  examples/FinRL_PortfolioOptimization_PPO_IntrinsicReward_Ablation.ipynb \
  --ExecutePreprocessor.timeout=-1
```

For the formal 6 variants $\times$ 3 seeds experiment, change `FINRL_INTRINSIC_RUN_MODE=full`.

```
[26]: best_rows = test_model_summary.loc[
        test_model_summary.groupby("commission_name")["sharpe_mean"].idxmax()
    ]
print("Highest mean PPO Sharpe by commission:")
for _, row in best_rows.iterrows():
    print(
        f" {row['commission_name']}: {row['variant']} ",
        f"n={int(row['seed_count'])}, ",
        f"Sharpe={row['sharpe_mean']:.3f}  $\pm$  {row['sharpe_std']:.3f}",
        sep="",
    )
print("Training cache hits:", f"{int(run_table['cache_hit'].sum())}/\n↪{len(run_table)}")
print("Treat differences smaller than cross-seed variability cautiously.")
```

Highest mean PPO Sharpe by commission:

with_fee: combined n=3, Sharpe=0.896 $\pm$ 0.162

Training cache hits: 13/18

Treat differences smaller than cross-seed variability cautiously.